

[shell] AI 辅助编写 Shell 脚本

前置知识

在学习使用 ChatGPT 编写 Shell 脚本之前，您需要了解以下知识点：

1. 基本计算机操作：您应该熟悉基本的计算机操作，例如创建和管理文件、文件夹，以及使用命令行界面。
2. 操作系统：了解 Linux 和 Unix 系统的基本概念，如文件系统结构、进程管理、权限控制等。
3. 命令行基础：熟悉基本的命令行操作，例如使用 `ls`、`cd`、`cp`、`mv`、`rm` 等常用命令。
4. Shell 基础知识：理解 Shell 的基本概念，例如 Bash、Zsh 等常见 Shell 的区别，以及 Shell 的作用。
5. 编程基础：具备基本的编程知识，如变量、循环、条件语句、函数等。

练习题

习题一 询问和探索技术方案

询问如何监控机器的 Cpu 和内存使用情况

习题二 批量文件重命名

编写一个脚本，批量重命名某个目录下的所有文件，例如将所有 `.txt` 文件的扩展名更改为 `.log`。

习题三 日志 or 数据备份

创建一个 Shell 脚本，定期对指定目录或文件进行压缩打包，并将其移动到备份存储位置。

习题四 从日志中提取特定信息

编写一个脚本，从日志文件中提取特定信息，例如统计某个时间段内的错误数量。下面是日志文件里的一段内容

```
2023-04-12 10:00:02 [INFO] Server started.
```

```
2023-04-12 10:05:10 [ERROR] Database connection failed.
```

```
2023-04-12 10:05:30 [ERROR] User authentication failed.
```

```
2023-04-12 10:10:45 [INFO] User login successful.
```

```
2023-04-12 10:15:12 [ERROR] File not found.
```

```
2023-04-12 10:20:55 [WARNING] Low disk space.
```

```
2023-04-12 10:30:00 [ERROR] Database connection failed.
```

参考答案

习题一 询问技术方案

第一轮提示

提示

在 Linux (CentOS7) 中如何监控机器的 cpu 和内存使用情况

答

在 CentOS 7 系统中，可以使用以下命令和工具来监控机器的 CPU 和内存使用情况：

1. **top** 命令：这是一个实时显示系统中各个进程的资源占用状况的实用工具。在终端中输入 **top** 并按回车键，即可查看 CPU 和内存使用情况。按 **q** 键退出。
2. **htop** 命令：与 **top** 类似，但更加强大且易于使用。首先，通过以下命令安装 **htop**：

```
sudo yum install epel-release -y  
sudo yum install htop -y
```

安装完成后，输入 **htop** 并按回车键。按 **q** 键退出。

1. **vmstat** 命令：用于报告有关虚拟内存统计信息。在终端中输入 **vmstat** 并按回车键，即可查看 CPU 和内存使用情况。例如，可以输入 **vmstat 5** 每隔 5 秒刷新一次数据。
2. **free** 命令：用于显示系统中物理和交换内存的使用情况。输入 **free -h** 可以查看易读的内存使用信息。
3. **sar** 命令：属于 **sysstat** 工具包的一部分，用于收集和报告系统活动信息。首先安装 **sysstat**：

```
sudo yum install sysstat -y
```

安装完成后，可以使用以下命令查看 CPU 使用情况：**sar -u**

1. **iostat** 命令：也属于 **sysstat** 工具包，用于监控系统输入输出设备的负载。安装 **sysstat** 后，输入 **iostat** 并按回车键，即可查看 CPU 和磁盘使用情况。
2. **ps** 命令：用于显示系统中当前活动的进程信息。可以使用 **ps aux** 命令查看所有进程的 CPU 和内存使用情况。

3. **glances** 工具：这是一个跨平台的监控工具，提供有关 CPU、内存、磁盘、网络等多方面的信息。安装方法：

```
sudo yum install epel-release -y
sudo yum install glances -y
```

安装完成后，输入 **glances** 并按回车键。

这些工具和命令可以帮助您实时监控 CentOS 7 系统的 CPU 和内存使用情况。选择适合您需求的工具，保持对系统性能的关注。

反馈

这个时候我们可以继续追问，假设你已经开始使用 top 命令开始监控内存和 cpu 的使用情况。但又看不太懂 top 命令给出的输出，这个时候就可以让 GPT 来帮忙解释

第二轮提示

提示

我执行了 top 命令，得到了如下结果

```
top - 10:49:12 up 8 min, 0 users, load average: 0.00, 0.00, 0.00
```

```
Tasks: 5 total, 1 running, 4 sleeping, 0 stopped, 0 zombie
```

```
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
```

```
MiB Mem : 31980.9 total, 31712.2 free, 143.5 used, 125.2 buff/cache
```

```
MiB Swap: 8192.0 total, 8192.0 free, 0.0 used. 31520.3 avail Mem
```

```
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND
```

```
9 root 20 0 2176 392 0 S 0.3 0.0 0:00.01 init
```

```
1 root 20 0 1816 1188 1108 S 0.0 0.0 0:00.02 init
```

```
8 root 20 0 2176 380 0 S 0.0 0.0 0:00.00 init
```

```
10 root 20 0 6072 5012 3344 S 0.0 0.0 0:00.02 bash
```

```
61 root 20 0 7784 3584 2984 R 0.0 0.0 0:00.04 top
```

请帮我解释一下上述内容

答

top 命令是 Linux 系统中的一个实时性能监控工具，它能够实时显示系统中各个进程的资源占用情况。下面我来解释一下你的 **top** 命令结果。

1. 第一行：系统时间、运行时间、当前用户数量和系统负载。

- 10:49:12: 表示当前系统时间。
- up 8 min: 表示系统已经运行了 8 分钟。
- 0 users: 表示当前有 0 个用户登录系统。
- load average: 0.00, 0.00, 0.00: 表示过去 1 分钟、5 分钟和 15 分钟的系统平均负载。

2. 第二行：进程概览。

- Tasks: 当前系统的进程总数。
- 5 total: 总共有 5 个进程。
- 1 running: 1 个进程正在运行。
- 4 sleeping: 4 个进程处于休眠状态。
- 0 stopped: 0 个进程被停止。
- 0 zombie: 0 个僵尸进程。

3. 第三行：CPU 使用情况。

- %Cpu(s): CPU 使用率。
- 0.0 us: 用户态占用 CPU 百分比。
- 0.0 sy: 内核态占用 CPU 百分比。
- 0.0 ni: 用于用户态优先级调整的进程占用 CPU 百分比。
- 100.0 id: 空闲 CPU 百分比。
- 0.0 wa: 等待 I/O 完成的 CPU 百分比。
- 0.0 hi: 处理硬件中断的 CPU 百分比。
- 0.0 si: 处理软件中断的 CPU 百分比。
- 0.0 st: 虚拟机偷取的 CPU 百分比（不常见）。

4. 第四行: 内存使用情况。

- MiB Mem: 物理内存。
- 31980.9 total: 总共有 31980.9 MiB 内存。
- 31712.2 free: 空闲的内存为 31712.2 MiB。
- 143.5 used: 已使用的内存为 143.5 MiB。
- 125.2 buff/cache: 缓冲区和缓存占用的内存为 125.2 MiB。

5. 第五行: 交换分区使用情况。

- MiB Swap: 交换分区。
- 8192.0 total: 总共有 8192.0 MiB 交换分区。
- 8192.0 free: 空闲的交换分区为 8192.0 MiB。
- 0.0 used: 已使用的交换分区为 0.0 MiB。

- 31520.3 avail Mem: 可用内存为 31520.3 MiB。

6. 后续行：各个进程的详细信息，包括进程 ID、

反馈

可以看到，GPT 可以作为你学习过程中的导师。不过这个答案展示地不是很全。遇到答案中断的情况，你可以立刻提示“继续”来让 GPT 接着生成内容

第三轮提示

提示

继续

答

用户、优先级、虚拟内存、实际内存、共享内存、状态、CPU 使用率、内存使用率、运行时间和命令名称。

以第一个进程为例：

- PID：进程 ID，唯一标识一个进程。这里的进程 ID 为 9。
- USER：进程的拥有者。这里的拥有者为 root。
- PR：进程的优先级。这里的优先级为 20。
- NI：进程的 nice 值，用于调整进程优先级。这里的 nice 值为 0。
- VIRT：进程占用的虚拟内存大小，单位为 KiB。这里的虚拟内存大小为 2176 KiB。
- RES：进程占用的实际内存大小，单位为 KiB。这里的实际内存大小为 392 KiB。
- SHR：进程使用的共享内存大小，单位为 KiB。这里的共享内存大小为 0 KiB。

- S: 进程的状态。S 表示休眠状态 (Sleeping)。
- %CPU: 进程占用的 CPU 使用率。这里的 CPU 使用率为 0.3%。
- %MEM: 进程占用的内存使用率。这里的内存使用率为 0.0%。
- TIME+: 进程运行的累计时间。这里的累计时间为 0:00.01。
- COMMAND: 进程对应的命令名称。这里的命令名称为 init。

其他进程的信息解释类似，你可以参照上述解释来理解每个进程的资源占用情况。

反馈

可以看到其实只要我们目前学的不是很新（21 年 9 月后）的技术，我们很多时候都可以将 GPT 作为一个出色的导师。

习题二 批量文件重命名

第一轮提示

提示

我要写一个 shell 脚本，他要完成以下功能

1. 从命令行接收一个参数（目录路径）并检查他的有效性
2. 将指定路径下以 .txt 结尾的文件全部改为 .log 结尾

编写过程中请加上详细的注释

答

下面是一个可以实现您要求功能的 shell 脚本。每一部分的功能都有详细的注释说明。

```
#!/bin/bash

# 这是一个可以将目录下所有.txt 文件扩展名更改为.log 的 shell 脚本

# 检查命令行参数的数量，如果不等于 1，则显示错误信息并退出脚本
if [ "$#" -ne 1 ]; then
    echo "错误：请输入一个目录路径作为参数"
    exit 1
fi

# 将命令行参数赋值给变量 directory
directory=$1

# 检查目录是否存在，如果不存在则显示错误信息并退出脚本
if [ ! -d "$directory" ]; then
    echo "错误：$directory 不是一个有效的目录"
    exit 1
fi

# 使用 find 命令查找指定路径下的所有.txt 文件，并将结果存储在变量 file_list 中
file_list=$(find "$directory" -type f -name "*.txt")

# 遍历所有.txt 文件
for file in $file_list; do
    # 获取不带扩展名的文件名，使用 basename 命令将.txt 从文件名中移除
    filename=$(basename "$file" .txt)

    # 获取文件所在的目录，使用 dirname 命令
    file_directory=$(dirname "$file")

    # 将.txt 扩展名更改为.log，并在原目录下保存新文件
    mv "$file" "$file_directory/$filename.log"

    # 输出已更改的文件名，方便用户查看
    echo "已将 $file 更改为 $file_directory/$filename.log"
done
```

```
# 脚本执行完毕，输出提示信息  
echo "脚本执行完毕，所有.txt 文件已更改为.log"
```

将此脚本保存为一个文件，例如 `change_extension.sh`，并通过命令 `chmod +x change_extension.sh` 使其可执行。然后您可以使用如下命令调用该脚本：

```
./change_extension.sh /path/to/your/directory
```

反馈

这个时候，应当将 GPT 给我们的 shell 脚本复制到虚拟机中进行测试。你也可以让 ChatGPT 自动帮你编写测试用例

第二轮提示

提示

请设计或编写一些测试用例以确保这个 shell 脚本的健壮性。

答

为确保这个 shell 脚本的健壮性，我们可以设计以下测试用例：

1. 测试一个存在的目录，其中包含多个以.txt 结尾的文件。
2. 测试一个存在的目录，但不包含任何以.txt 结尾的文件。
3. 测试一个存在的目录，其中包含子目录，子目录中包含以.txt 结尾的文件。
4. 测试一个存在的目录，其中包含以.txt 结尾的文件和以其他扩展名结尾的文件。
5. 测试一个不存在的目录。
6. 测试一个文件而非目录。

7. 不提供参数，即不输入目录。

8. 提供多个参数。

要执行这些测试用例，首先需要创建一些测试文件和目录，然后运行脚本并观察输出结果。以下是一个简单的测试脚本，用于设置和执行前四个测试用例：

```
#!/bin/bash

# 创建测试目录和文件
mkdir -p test_case1
touch test_case1/file1.txt test_case1/file2.txt

mkdir -p test_case2
touch test_case2/file1.log test_case2/file2.log

mkdir -p test_case3/subdir
touch test_case3/subdir/file1.txt test_case3/subdir/file2.txt

mkdir -p test_case4
touch test_case4/file1.txt test_case4/file2.log

# 运行脚本并检查输出
echo "测试用例 1: 存在的目录，包含多个以.txt 结尾的文件"
./change_extension.sh test_case1

echo "测试用例 2: 存在的目录，不包含以.txt 结尾的文件"
./change_extension.sh test_case2

echo "测试用例 3: 存在的目录，包含子目录，子目录中包含以.txt 结尾的文件"
./change_extension.sh test_case3

echo "测试用例 4: 存在的目录，包含以.txt 结尾的文件和以其他扩展名结尾的文件"
./change_extension.sh test_case4

# 删除测试目录和文件
```

```
rm -rf test_case1 test_case2 test_case3 test_case4
```

对于后四个测试用例，您可以手动执行脚本并检查输出：

```
# 测试用例 5：不存在的目录
./change_extension.sh non_existent_directory

# 测试用例 6：一个文件而非目录
touch test_file.txt
./change_extension.sh test_file.txt
rm test_file.txt

# 测试用例 7：不提供参数
./change_extension.sh

# 测试用例 8：提供多个参数
./change_extension.sh test_case1 test_case2
```

通过观察输出结果和检查目录中的文件，您可以确认脚本是否正确处理了这些测试用例。

反馈

你可以编写自己的测试用例，或者使用 ChatGPT 给出的测试方法进行测试。如果遇到不理解的地方，可以继续向 GPT 提问以搞清楚问题，例如该 shell 脚本是否会递归修改子目录的文件。

第三轮提示

提示

这份脚本会递归修改子目录里的文件吗？哪几行代码决定了这个行为？

答

是的，这个脚本会递归修改子目录里的.txt文件。这个行为是由以下代码行决定的：

```
# 使用 find 命令查找指定路径下的所有.txt文件，并将结果存储在变量 file_list 中
file_list=$(find "$directory" -type f -name "*.txt")
```

在这行代码中，我们使用 **find** 命令来查找指定路径下的所有.txt文件。**find** 命令会递归搜索指定目录及其子目录中的文件。这里的 **-type f** 参数表示我们只关心文件类型，而 **-name "*.txt"** 参数表示我们只关心以.txt结尾的文件。

由于 **find** 命令会递归搜索子目录，所以在后面的循环中，我们可以处理所有匹配的.txt文件，包括子目录中的文件。

反馈

如果你已经对这个脚本的工作原理进行了详尽的测试，并且确认其可用于生产环境，那么你可以考虑将其应用于实际生产环境中。但是有经验的程序员会意识到，这个脚本在功能和测试方面仍然需要添加一个关键层面，即权限检查。

第四轮提示

提示

你是不是没有考虑目录和文件的权限问题

答

您是对的，我的脚本确实没有考虑目录和文件的权限问题。为了让脚本更健壮，我们可以在脚本中检查目录和文件的读写权限。

以下是修改后的脚本，增加了权限检查：

```
#!/bin/bash

# 这是一个可以将目录下所有.txt 文件扩展名更改为.log 的 shell 脚本，并检查目录和文件权限

# 检查命令行参数的数量，如果不等于 1，则显示错误信息并退出脚本
if [ "$#" -ne 1 ]; then
    echo "错误：请输入一个目录路径作为参数"
    exit 1
fi

# 将命令行参数赋值给变量 directory
directory=$1

# 检查目录是否存在，如果不存在则显示错误信息并退出脚本
if [ ! -d "$directory" ]; then
    echo "错误：$directory 不是一个有效的目录"
    exit 1
fi

# 检查目录是否可读，如果不可读则显示错误信息并退出脚本
if [ ! -r "$directory" ]; then
    echo "错误：您没有读取 $directory 的权限"
    exit 1
fi

# 使用 find 命令查找指定路径下的所有.txt 文件，并将结果存储在变量 file_list 中
file_list=$(find "$directory" -type f -name "*.txt")

# 遍历所有.txt 文件
for file in $file_list; do
    # 检查文件是否可读，如果不可读则跳过此文件并继续处理下一个文件
    if [ ! -r "$file" ]; then
        echo "警告：您没有读取 $file 的权限，跳过此文件"
        continue
    fi

    # 检查文件是否可写，如果不可写则跳过此文件并继续处理下一个文件
```

```
if [ ! -w "$file" ]; then
    echo "警告：您没有写入 $file 的权限，跳过此文件"
    continue
fi

# 获取不带扩展名的文件名，使用 basename 命令将 .txt 从文件名中移除
filename=$(basename "$file" .txt)

# 获取文件所在的目录，使用 dirname 命令
file_directory=$(dirname "$file")

# 将 .txt 扩展名更改为 .log，并在原目录下保存新文件
mv "$file" "$file_directory/$filename.log"

# 输出已更改的文件名，方便用户查看
echo "已将 $file 更改为 $file_directory/$filename.log"
done

# 脚本执行完毕，输出提示信息
echo "脚本执行完毕，所有 .txt 文件已更改为 .log"
```

在这个修改后的脚本中，我们在处理目录和文件之前分别检查了它们的读取权限和写入权限。如果缺少相应的权限，脚本将显示警告信息并跳过该文件，继续处理下一个文件。这样可以确保脚本在遇到权限问题时能够正常运行并提供有用的反馈。

反馈

我们认为这个脚本已经完全满足需求。

习题三 日志 or 数据备份

第一轮提示

提示

帮我写一个 Shell 脚本。这个脚本要定期对指定目录进行压缩打包，并将其移动到备份存储位置

答

当然可以。以下是一个简单的 Shell 脚本，用于定期压缩指定目录并将其移动到备份存储位置。这个脚本使用 **tar** 命令进行压缩，并使用 **mv** 命令将压缩文件移动到目标目录。我们将使用 **cron** 来定期运行此脚本。

首先，创建一个名为 **backup_script.sh** 的文件，并在其中编写以下脚本：

```
#!/bin/bash

# 定义需要备份的源目录和目标备份目录
SOURCE_DIR="/path/to/your/source/directory"
TARGET_DIR="/path/to/your/backup/directory"

# 获取当前日期和时间
DATE=$(date +%Y-%m-%d_%H-%M-%S)

# 创建压缩文件名
ARCHIVE_NAME="backup_${DATE}.tar.gz"

# 使用 tar 命令压缩源目录
tar -czvf "${TARGET_DIR}/${ARCHIVE_NAME}" "${SOURCE_DIR}"

# 检查压缩是否成功
if [ $? -eq 0 ]; then
    echo "备份成功: ${TARGET_DIR}/${ARCHIVE_NAME}"
```

```
else
    echo "备份失败"
fi
```

确保脚本具有可执行权限：

```
chmod +x backup_script.sh
```

现在，你可以手动运行脚本，以确保其按预期工作：

```
./backup_script.sh
```

要定期运行此脚本，可以将其添加到 **cron** 中。打开终端并输入以下命令以编辑 **crontab**：

```
crontab -e
```

现在，添加一行以定义定期运行脚本的时间。例如，以下行将每天凌晨 1 点执行脚本：

```
0 1 * * * /path/to/your/backup_script.sh
```

保存并退出编辑器。现在，你的脚本将按计划定期运行，压缩指定目录并将其移动到备份存储位置。

反馈

这个解决方案是很健全的，GPT 还建议你用 **crontab** 来做周期执行调度。并给出了 **crontab** 里的调度脚本。需要注意的是，之前的脚本里面我们做了权限检查的纠正，这是因为我们之前的脚本是工具性质的。现在所编写的脚本，在使用方式上更偏向运

维。这个时候，通常不会再对用户输入进行严格限制，而是在运行前缺点特定目录符合输入要求就可以了。

习题四 从日志中提取特定信息

第一轮提示

提示

编写一个 Shell 脚本，从日志文件中提取特定信息，例如统计某个时间段内的错误数量、不同类型的错误数量等。

下面是日志文件里的一段内容

```
2023-04-12 10:00:02 [INFO] Server started.
```

```
2023-04-12 10:05:10 [ERROR] Database connection failed.
```

```
2023-04-12 10:05:30 [ERROR] User authentication failed.
```

```
2023-04-12 10:10:45 [INFO] User login successful.
```

```
2023-04-12 10:15:12 [ERROR] File not found.
```

```
2023-04-12 10:20:55 [WARNING] Low disk space.
```

```
2023-04-12 10:30:00 [ERROR] Database connection failed.
```

答

假设日志文件名为 `log_file.log`，我们可以编写一个 Shell 脚本来提取所需的信息。

以下是一个脚本，它接受开始时间、结束时间和日志级别作为输入参数，然后统计给定时间段内相应日志级别的数量。

```
#!/bin/bash
```

```

# Usage: ./extract_log_info.sh "2023-04-12 10:00:00" "2023-04-12 10:30:00" "ERROR"

start_time="$1"
end_time="$2"
log_level="$3"

if [ -z "$start_time" ] || [ -z "$end_time" ] || [ -z "$log_level" ]; then
    echo "Usage: $0 start_time end_time log_level"
    exit 1
fi

start_time_epoch=$(date -d "$start_time" +%s)
end_time_epoch=$(date -d "$end_time" +%s)

if [ "$start_time_epoch" -gt "$end_time_epoch" ]; then
    echo "Error: Start time must be earlier than end time."
    exit 1
fi

log_count=$(awk -v start="$start_time_epoch" -v end="$end_time_epoch" -v
level="[$log_level]" \
'BEGIN {
    count = 0
}
{
    timestamp = $1 " " " $2
    timestamp_epoch = mktime(gensub(/[:-:]/, " ", "g", timestamp))
    log_level = $3

    if (timestamp_epoch >= start && timestamp_epoch <= end && log_level == level) {
        count++
    }
}
END {
    print count
}' log_file.log)

```

```
echo "Number of $log_level logs between $start_time and $end_time: $log_count"
```

将上述代码保存为 **extract_log_info.sh** 并使其可执行：

```
chmod +x extract_log_info.sh
```

使用示例：

```
./extract_log_info.sh "2023-04-12 10:00:00" "2023-04-12 10:30:00" "ERROR"
```

反馈

现在 GPT 给出了一段 Shell 脚本，如果你觉得可读性不好，或者较难理解其中的内容，那么你可以让他加上注释。

第二轮提示

提示

请使用中文给之前的 Shell 脚本加上详细的注释

答

```
#!/bin/bash

# 使用方法: ./extract_log_info.sh "2023-04-12 10:00:00" "2023-04-12 10:30:00" "ERROR"

# 将输入参数存储在变量中
start_time="$1"
end_time="$2"
log_level="$3"
```

```
# 检查是否提供了所有必需的输入参数
if [ -z "$start_time" ] || [ -z "$end_time" ] || [ -z "$log_level" ]; then
    echo "用法: $0 开始时间 结束时间 日志级别"
    exit 1
fi

# 将开始时间和结束时间转换为 Unix 时间戳（以秒为单位）
start_time_epoch=$(date -d "$start_time" +%s)
end_time_epoch=$(date -d "$end_time" +%s)

# 检查开始时间是否早于结束时间
if [ "$start_time_epoch" -gt "$end_time_epoch" ]; then
    echo "错误: 开始时间必须早于结束时间。"
    exit 1
fi

# 使用 awk 命令分析日志文件并统计给定时间段内特定日志级别的数量
log_count=$(awk -v start="$start_time_epoch" -v end="$end_time_epoch" -v
level="[$log_level]" \
'BEGIN {
    count = 0
}
{
    # 从日志中提取时间戳和日志级别
    timestamp = $1 " " $2
    timestamp_epoch = mktime(gensub(/[-:]/, " ", "g", timestamp))
    log_level = $3

    # 检查时间戳是否在给定范围内，日志级别是否与输入参数匹配
    if (timestamp_epoch >= start && timestamp_epoch <= end && log_level == level) {
        count++
    }
}
END {
    # 输出计数结果
```

```
    print count
}' log_file.log)

# 显示给定时间段内特定日志级别的数量
echo "在 $start_time 和 $end_time 之间的 $log_level 日志数量: $log_count"
```

反馈

现在，我们可以看到这个脚本是把要处理的日志文件名写死在脚本里的。现在，我们希望能够通过参数来控制脚本要分析哪个日志文件。

第三轮提示

提示

我看到 `log_file.log` 作为要分析的日志文件是写死在脚本里的，我希望能够通过命令行传递参数的方式来指定我想分析哪个文件

答

可以修改脚本，使其接受日志文件名作为一个额外的输入参数。以下是更新后的脚本：

```
#!/bin/bash

# Usage: ./extract_log_info.sh "2023-04-12 10:00:00" "2023-04-12 10:30:00" "ERROR"
"log_file.log"

start_time="$1"
end_time="$2"
log_level="$3"
log_file="$4"
```

```

if [ -z "$start_time" ] || [ -z "$end_time" ] || [ -z "$log_level" ] || [ -z "$log_file" ];
then
    echo "用法: $0 开始时间 结束时间 日志级别 日志文件"
    exit 1
fi

start_time_epoch=$(date -d "$start_time" +%s)
end_time_epoch=$(date -d "$end_time" +%s)

if [ "$start_time_epoch" -gt "$end_time_epoch" ]; then
    echo "错误: 开始时间必须早于结束时间。"
    exit 1
fi

log_count=$(awk -v start="$start_time_epoch" -v end="$end_time_epoch" -v
level="[$log_level]" \
'BEGIN {
    count = 0
}
{
    timestamp = $1 " " $2
    timestamp_epoch = mktime(gensub(/[:-:]/, " ", "g", timestamp))
    log_level = $3

    if (timestamp_epoch >= start && timestamp_epoch <= end && log_level == level) {
        count++
    }
}
END {
    print count
}' "$log_file")

echo "在 $start_time 和 $end_time 之间的 $log_level 日志数量: $log_count"

```

现在，您可以通过以下命令行传递参数来指定要分析的日志文件：

```
./extract_log_info.sh "2023-04-12 10:00:00" "2023-04-12 10:30:00" "ERROR"  
"your_log_file.log"
```

将 **your_log_file.log** 替换为您想分析的日志文件名。

反馈

现在，我们可以在自己的 linux 进行测试。如果测试通过，那么可以决定将它用在生产场景。

```
root@dengziqui:~# ./extract_log_info.sh "2023-04-12 10:00:00" "2023-04-12 10:30:00" "ERROR" ./log.txt  
在 2023-04-12 10:00:00 和 2023-04-12 10:30:00 之间的 ERROR 日志数量: 4
```